Section A – Concept Application

S1 SCENARIO You are designing the database for a food delivery application. It stores restaurant details, menu items linked to each restaurant, and customer orders. You must decide which columns need NOT NULL constraints, which serve as PRIMARY KEYs, and which link tables via FOREIGN KEYs.

Question 1: Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

```
CREATE TABLE restaurants (
   restaurant_id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   city VARCHAR(60)
);

CREATE TABLE menu_items (
   item_id INT PRIMARY KEY AUTO_INCREMENT,
   restaurant_id INT NOT NULL,
   item_name VARCHAR(100) NOT NULL,
   price DECIMAL(8,2) NOT NULL,
   FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);
```

S2 SCENARIO You are managing the database for a food delivery platform. The operations team asks you to update the status of all orders placed more than 7 days ago that are still marked 'Pending' to 'Cancelled', and to permanently delete menu items marked 'Unavailable' from restaurants that have closed.

Question 2: Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?

```
UPDATE orders
SET status = 'Cancelled'
WHERE order_date < DATE_SUB(CURDATE(), INTERVAL 7 DAY)
  AND status = 'Pending';

DELETE mi
FROM menu_items AS mi
JOIN restaurants AS r ON r.restaurant_id = mi.restaurant_id
```

```
WHERE mi.status = 'Unavailable'
  AND r.status = 'Closed';
```

S3 SCENARIO You are building a revenue dashboard for a food delivery startup. The management wants total revenue, average order value, and number of orders per restaurant — but only for restaurants that have processed more than 20 orders in the current month.

Question 3: Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results

```
SELECT restaurant_id,
    SUM(total_amount) AS total_revenue,
    AVG(total_amount) AS average_order_value,
    COUNT(*) AS number_of_orders
FROM orders
WHERE order_date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
  AND order_date < DATE_ADD(DATE_FORMAT(CURDATE(), '%Y-%m-01'), INTERVAL 1 MONTH)
GROUP BY restaurant_id
HAVING COUNT(*) > 20;
```

S4 SCENARIO You are developing an order history page. The orders table has order_id, customer_id, restaurant_id, and total_amount. The restaurants table has restaurant_id and name. The page must show order ID, restaurant name, and amount for every order — including any that reference a restaurant no longer in the table.

Question 4: Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table.

```
SELECT o.order_id, r.name AS restaurant_name, o.total_amount
FROM orders AS o
LEFT JOIN restaurants AS r
   ON o.restaurant_id = r.restaurant_id;
```

S5 SCENARIO You are building the analytics layer for a food delivery platform. A senior developer proposes a VIEW called top_restaurants showing restaurant name, total orders, and average rating — hiding raw ID columns from the front-end team.

Question 5: Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

```
CREATE VIEW top_restaurants AS
SELECT r.name AS restaurant_name,
    COUNT(o.order_id) AS total_orders,
    AVG(o.rating) AS average_rating
FROM restaurants AS r
LEFT JOIN orders AS o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name;
```

S6 SCENARIO You are implementing the payment module for a food delivery app. When a customer places an order, the system must deduct the amount from the customer's wallet balance and record the payment simultaneously. During load testing, the wallet deduction completes but the payment record insert occasionally fails, leaving balances inconsistent.

Question 6: Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

```
START TRANSACTION;

UPDATE customers
SET wallet_balance = wallet_balance - 500
WHERE customer_id = 1
  AND wallet_balance >= 500;

INSERT INTO payments (customer_id, amount, payment_status)
VALUES (1, 500, 'Successful');

COMMIT;
```

Section B – Practical Coding Tasks

Task 1: Design and populate restaurant database

Create a structured schema for a food delivery system and populate it with sample data using DDL and DML commands.

Create a database named foodapp_db and a table named restaurants with columns: restaurant_id (INT, AUTO_INCREMENT, PRIMARY KEY), name (VARCHAR, NOT NULL), city (VARCHAR), cuisine_type (VARCHAR), and rating (DECIMAL(3,1)).

```
CREATE DATABASE EXISTS foodapp_db;
USE foodapp_db;

CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(60),
    cuisine_type VARCHAR(60),
    rating DECIMAL(3,1)
);
```

Insert at least 6 restaurant records spanning at least 3 different cities.

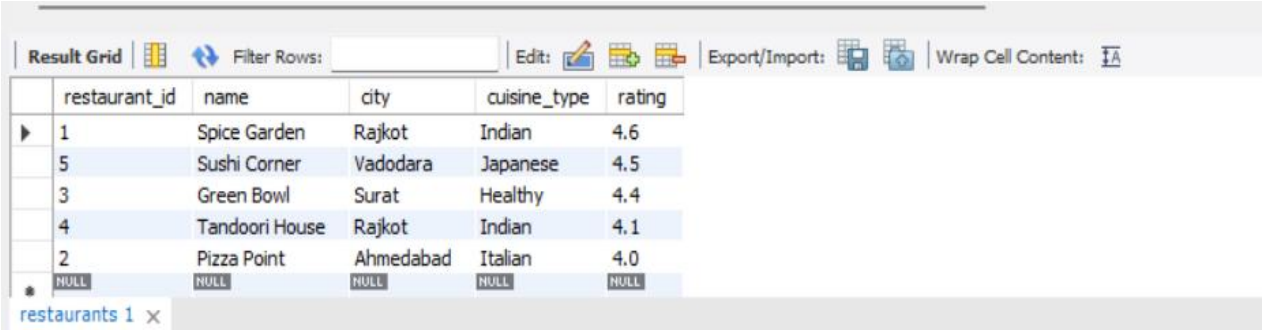
```
INSERT INTO restaurants (name, city, cuisine_type, rating) VALUES
('Spice Garden', 'Rajkot', 'Indian', 4.2),
('Pizza Point', 'Ahmedabad', 'Italian', 4.0),
('Green Bowl', 'Surat', 'Healthy', 4.4),
('Tandoori House', 'Rajkot', 'Indian', 4.1),
('Sushi Corner', 'Vadodara', 'Japanese', 4.5),
('Sweet Treats', 'Ahmedabad', 'Dessert', 3.9);
```

Update the rating of one restaurant and delete one restaurant record using a WHERE condition.

```
UPDATE restaurants
SET rating = 4.6
WHERE restaurant_id = 1;
DELETE FROM restaurants
WHERE restaurant_id = 6;
```

Write a SELECT query that retrieves all restaurants sorted by rating in descending order.

```
SELECT *
FROM restaurants
ORDER BY rating DESC;
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

restaurant_id	name	city	cuisine_type	rating
1	Spice Garden	Rajkot	Indian	4.6
5	Sushi Corner	Vadodara	Japanese	4.5
3	Green Bowl	Surat	Healthy	4.4
4	Tandoori House	Rajkot	Indian	4.1
2	Pizza Point	Ahmedabad	Italian	4.0
NULL	NULL	NULL	NULL	NULL

restaurants 1

Task 2: Aggregate revenue reporting

Write SELECT queries using aggregate functions and GROUP BY to produce a restaurant-level revenue summary from order data.

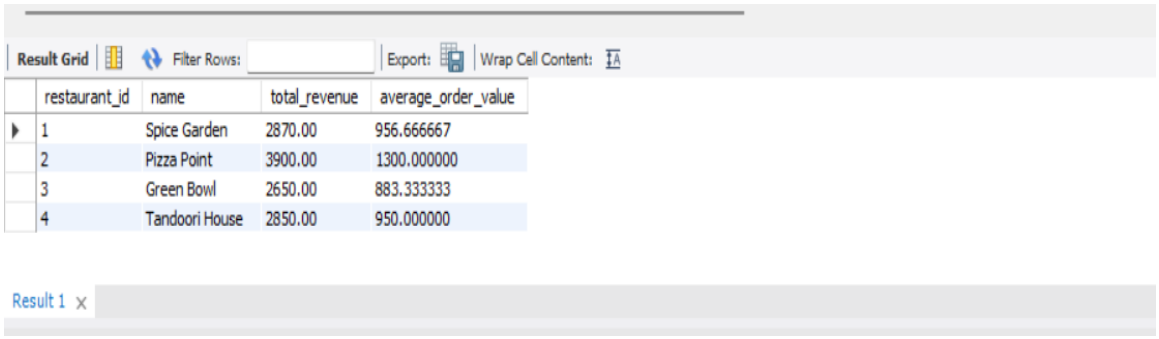
Create an orders table with columns: order_id, restaurant_id, customer_name, total_amount, and order_date. Insert at least 12 orders distributed across at least 4 restaurants.

```
USE foodapp_db;
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);
INSERT INTO orders (restaurant_id, customer_name, total_amount, order_date)
VALUES
(1, 'Asha', 650.00, '2026-10-01'),
(2, 'Bharat', 1200.00, '2026-10-02'),
(3, 'Chirag', 850.00, '2026-10-03'),
(4, 'Diya', 950.00, '2026-10-04'),
(1, 'Esha', 720.00, '2026-10-05'),
(2, 'Farhan', 1800.00, '2026-10-06'),
(3, 'Gita', 1100.00, '2026-10-07'),
(4, 'Harsh', 600.00, '2026-10-08'),
(1, 'Isha', 1500.00, '2026-10-09'),
```

```
(2, 'Jay', 900.00, '2026-10-10'),
(3, 'Kavya', 700.00, '2026-10-11'),
(4, 'Manav', 1300.00, '2026-10-12');
```

Write a query to show total revenue and average order value per restaurant.

```
SELECT r.restaurant_id, r.name,
SUM(o.total_amount) AS total_revenue,
AVG(o.total_amount) AS average_order_value
FROM restaurants AS r
JOIN orders AS o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name;
```



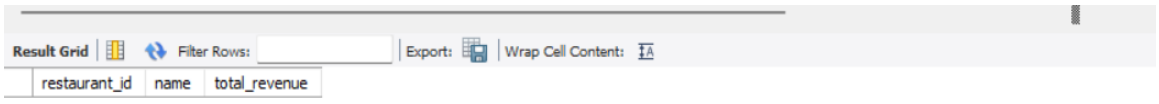
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_id	name	total_revenue	average_order_value
1	Spice Garden	2870.00	956.666667
2	Pizza Point	3900.00	1300.000000
3	Green Bowl	2650.00	883.333333
4	Tandoori House	2850.00	950.000000

Result 1 x

Write a query using HAVING to list only restaurants whose total revenue exceeds Rs 5,000.

```
SELECT r.restaurant_id, r.name,
       SUM(o.total_amount) AS total_revenue
FROM restaurants AS r
JOIN orders AS o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING SUM(o.total_amount) > 5000;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

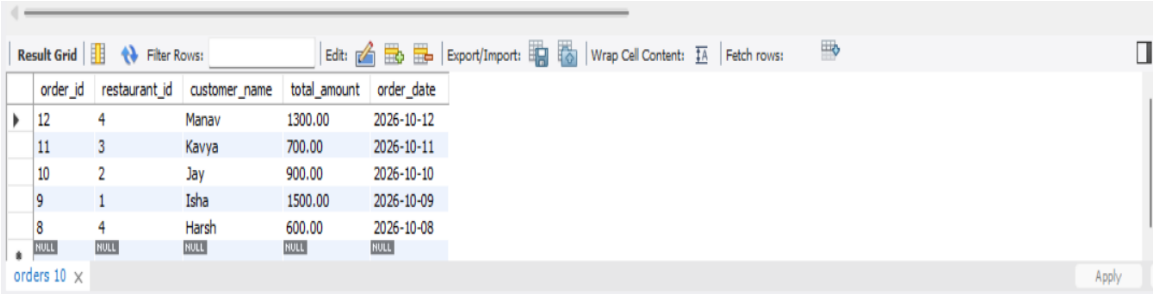
restaurant_id	name	total_revenue

Result 9 x

Write a query to retrieve the 5 most recent orders using ORDER BY and LIMIT.

```
SELECT *
FROM orders
```

```
ORDER BY order_date DESC, order_id DESC
LIMIT 5;
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

order_id	restaurant_id	customer_name	total_amount	order_date
12	4	Manav	1300.00	2026-10-12
11	3	Kavya	700.00	2026-10-11
10	2	Jay	900.00	2026-10-10
9	1	Isha	1500.00	2026-10-09
8	4	Harsh	600.00	2026-10-08
NULL	NULL	NULL	NULL	NULL

orders 10 | Apply

Task 3: JOINs and menu summary VIEW

Build a two-table schema for restaurants and menu items, write JOIN queries to combine data, and create a summary VIEW.

Create two tables: restaurants (restaurant_id, name, city) and menu_items (item_id, restaurant_id, item_name, price, category). Link them with a FOREIGN KEY on restaurant_id.

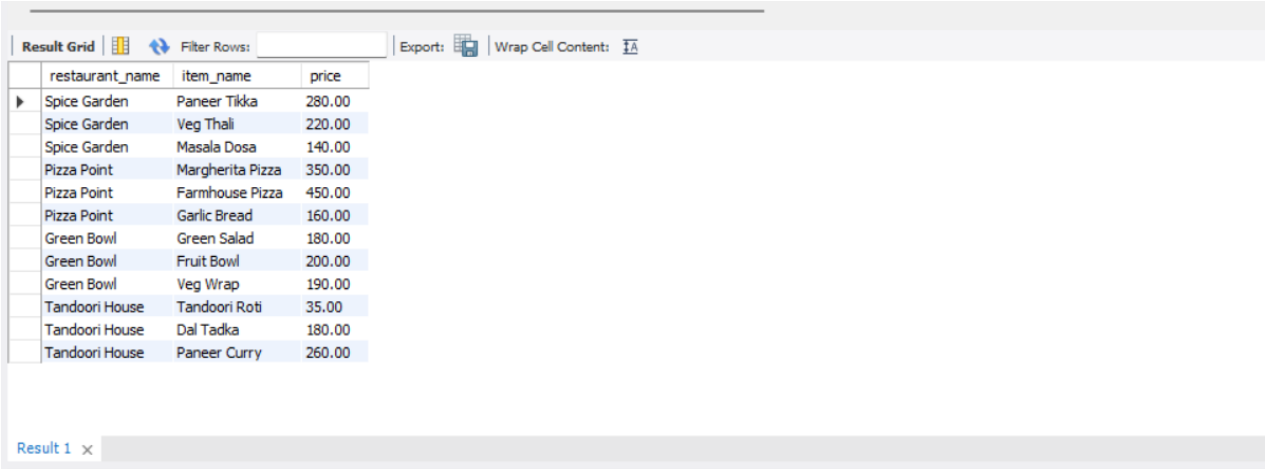
```
CREATE TABLE menu_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    price DECIMAL(8,2) NOT NULL,
    category VARCHAR(50),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);
```

Insert at least 5 restaurants and 12 menu items; at least one restaurant must have no menu items.

```
INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES
(1, 'Paneer Tikka', 280.00, 'Starter'),
(1, 'Veg Thali', 220.00, 'Main Course'),
(1, 'Masala Dosa', 140.00, 'Main Course'),
(2, 'Margherita Pizza', 350.00, 'Pizza'),
(2, 'Farmhouse Pizza', 450.00, 'Pizza'),
(2, 'Garlic Bread', 160.00, 'Starter'),
(3, 'Green Salad', 180.00, 'Healthy'),
(3, 'Fruit Bowl', 200.00, 'Healthy'),
(3, 'Veg Wrap', 190.00, 'Main Course'),
(4, 'Tandoori Roti', 35.00, 'Bread'),
(4, 'Dal Tadka', 180.00, 'Main Course'),
(4, 'Paneer Curry', 260.00, 'Main Course');
-- Restaurant 5 has no menu items, to test LEFT JOIN.
```

Write an INNER JOIN query listing restaurant name, item name, and price for all available items.

```
SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants AS r
INNER JOIN menu_items AS m ON r.restaurant_id = m.restaurant_id;
```



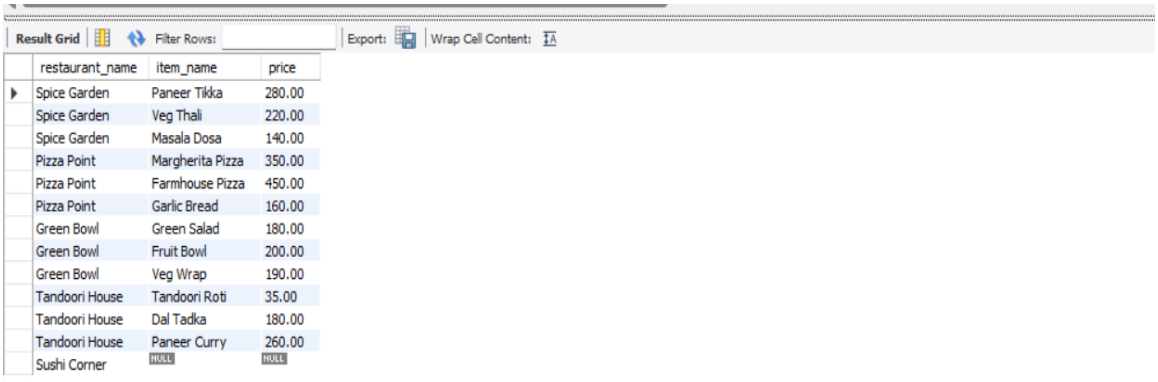
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_name	item_name	price
Spice Garden	Paneer Tikka	280.00
Spice Garden	Veg Thali	220.00
Spice Garden	Masala Dosa	140.00
Pizza Point	Margherita Pizza	350.00
Pizza Point	Farmhouse Pizza	450.00
Pizza Point	Garlic Bread	160.00
Green Bowl	Green Salad	180.00
Green Bowl	Fruit Bowl	200.00
Green Bowl	Veg Wrap	190.00
Tandoori House	Tandoori Roti	35.00
Tandoori House	Dal Tadka	180.00
Tandoori House	Paneer Curry	260.00

Result 1

Write a LEFT JOIN query so that restaurants with no items also appear in results, with NULL values in item columns.

```
SELECT r.name AS restaurant_name, m.item_name, m.price
FROM restaurants AS r
LEFT JOIN menu_items AS m ON r.restaurant_id = m.restaurant_id;
```



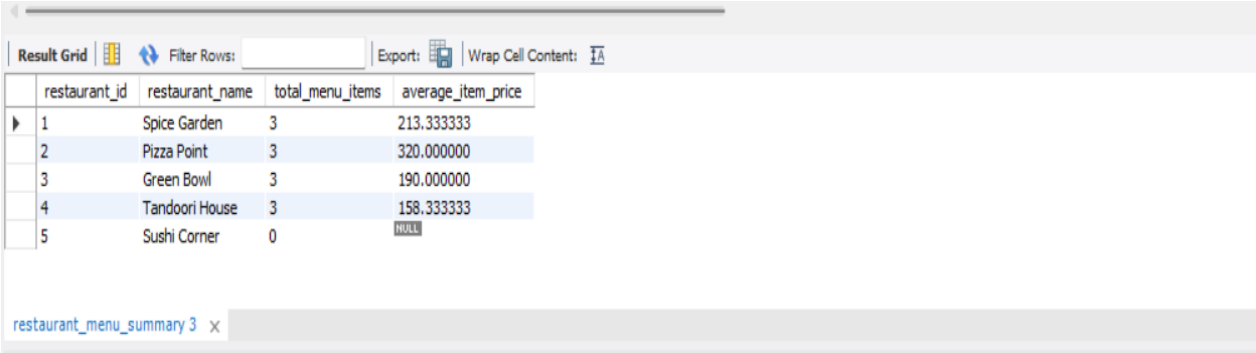
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_name	item_name	price
Spice Garden	Paneer Tikka	280.00
Spice Garden	Veg Thali	220.00
Spice Garden	Masala Dosa	140.00
Pizza Point	Margherita Pizza	350.00
Pizza Point	Farmhouse Pizza	450.00
Pizza Point	Garlic Bread	160.00
Green Bowl	Green Salad	180.00
Green Bowl	Fruit Bowl	200.00
Green Bowl	Veg Wrap	190.00
Tandoori House	Tandoori Roti	35.00
Tandoori House	Dal Tadka	180.00
Tandoori House	Paneer Curry	260.00
Sushi Corner	NULL	NULL

Create a VIEW named restaurant_menu_summary showing restaurant name, total number of menu items, and average item price grouped by restaurant.

```
CREATE VIEW restaurant_menu_summary AS
SELECT r.restaurant_id, r.name AS restaurant_name,
       COUNT(m.item_id) AS total_menu_items,
       AVG(m.price) AS average_item_price
FROM restaurants AS r
```

```
LEFT JOIN menu_items AS m ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.name;
SELECT * FROM restaurant_menu_summary;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_id	restaurant_name	total_menu_items	average_item_price
1	Spice Garden	3	213.333333
2	Pizza Point	3	320.000000
3	Green Bowl	3	190.000000
4	Tandoori House	3	158.333333
5	Sushi Corner	0	NULL

restaurant_menu_summary 3

Task 4: Stored procedure for order placement

Write a stored procedure to process a customer order with wallet deduction and full transaction control.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    wallet_balance DECIMAL(10,2)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    restaurant_id INT,
    amount DECIMAL(10,2)
);
INSERT INTO customers
VALUES (1, 500.00);
```

Create a stored procedure named place_order that accepts customer_id, restaurant_id, and amount as parameters.

Deduct the amount from the customer's wallet_balance in a customers table and insert a new record into the orders table.

Use a transaction block: if the customer's wallet_balance is insufficient, ROLLBACK everything; otherwise COMMIT.

```
DELIMITER //

CREATE PROCEDURE place_order(

    IN p_customer_id INT,

    IN p_restaurant_id INT,

    IN p_amount DECIMAL(10,2)

)

BEGIN

    DECLARE v_balance DECIMAL(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    START TRANSACTION;

    SELECT wallet_balance INTO v_balance

    FROM customers

    WHERE customer_id = p_customer_id

    FOR UPDATE;

    IF v_balance >= p_amount AND p_amount > 0 THEN

        UPDATE customers

        SET wallet_balance = wallet_balance - p_amount

        WHERE customer_id = p_customer_id;

        INSERT INTO orders (customer_id, restaurant_id, amount)

        VALUES (p_customer_id, p_restaurant_id, p_amount);

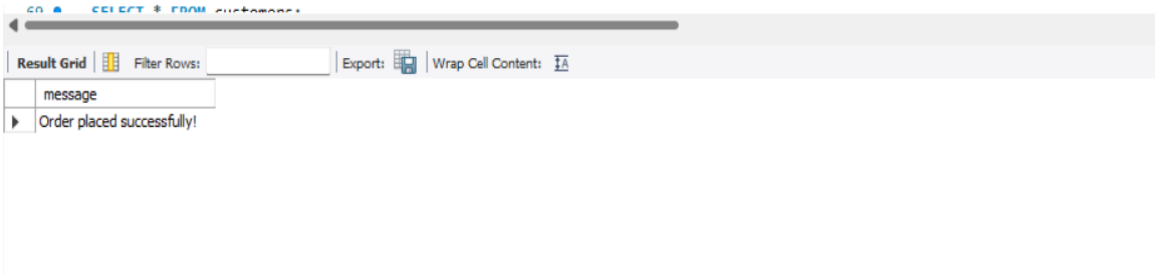
        COMMIT;

        SELECT 'Order placed successfully!' AS message;
```

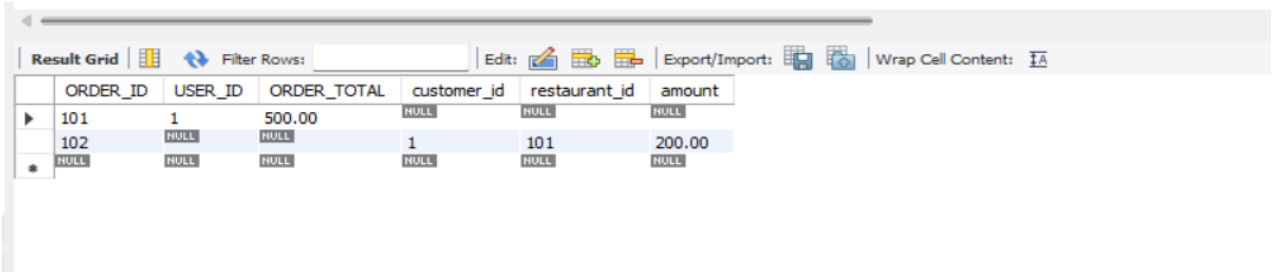
```
    ELSE
        ROLLBACK;
        SELECT 'Insufficient balance or invalid amount!' AS message;
    END IF;
END //
DELIMITER ;
CALL place_order(1, 1, 500.00);
```

Test with a valid order and an order exceeding wallet balance. Screenshot both outcomes from MySQL Workbench.

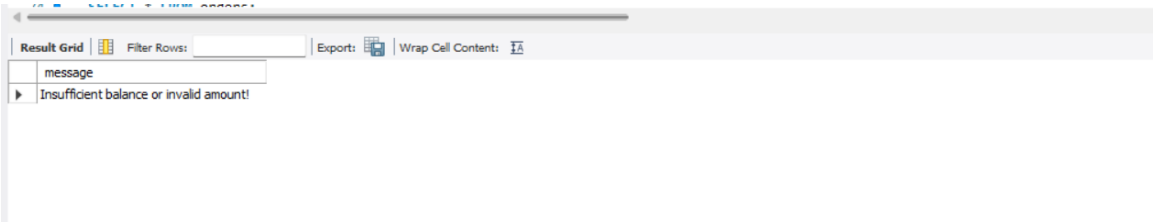
```
CALL place_order(1, 101, 200.00);
```




```
SELECT * FROM customers;
SELECT * FROM orders;
```



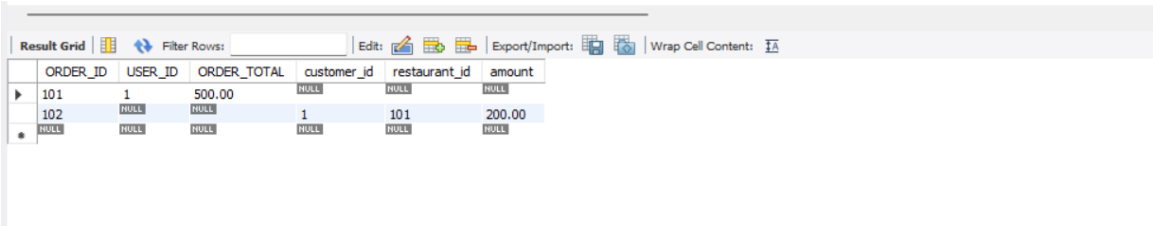
```
CALL place_order(1, 102, 400.00);
```



message
Insufficient balance or invalid amount!

```
SELECT * FROM customers;

SELECT * FROM orders;
```



ORDER_ID	USER_ID	ORDER_TOTAL	customer_id	restaurant_id	amount
101	1	500.00	NULL	NULL	NULL
102	NULL	NULL	1	101	200.00
NULL	NULL	NULL	NULL	NULL	NULL

Section C – Mini Capstone Project

Mini Project: Food Delivery Platform — Backend Database

Objective: Build the complete backend database for a Food Delivery application, combining DDL, DML, JOINs, Views, and PL/SQL into a single integrated .sql script.

Your project must:

Create three tables: restaurants (restaurant_id, name, city, cuisine_type, created_at), menu_items (item_id, restaurant_id, item_name, price, category) with a FOREIGN KEY on restaurant_id, and orders (order_id, customer_name, restaurant_id, item_id, quantity, total_amount, order_date) with a FOREIGN KEY on restaurant_id — all with appropriate PRIMARY KEY and NOT NULL constraints.

Populate with at least 3 restaurants, 4 menu categories, and 15 order records distributed across restaurants.

Create a VIEW named restaurant_sales_summary showing restaurant name, total number of orders, and total revenue grouped by restaurant.

Write a stored procedure named add_order that accepts customer_name, restaurant_id, item_id, and quantity, calculates total_amount from the menu item price, validates the restaurant exists (ROLLBACK if not), and inserts the order record.

Write a TRIGGER named after_order_insert that logs a record into an order_audit table (order_id, restaurant_id, action='INSERT', log_time=NOW()) immediately after every new order is created.

```
CREATE DATABASE IF NOT EXISTS food_delivery_capstone;
USE food_delivery_capstone;

CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(60) NOT NULL,
    cuisine_type VARCHAR(60) NOT NULL,
```

```
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE menu_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    category VARCHAR(50) NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    restaurant_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
    FOREIGN KEY (item_id) REFERENCES menu_items(item_id)
);

CREATE TABLE order_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    action VARCHAR(10) NOT NULL,
    log_time DATETIME NOT NULL
);

INSERT INTO restaurants (name, city, cuisine_type) VALUES
('Spice Garden', 'Rajkot', 'Indian'),
('Pizza Point', 'Ahmedabad', 'Italian'),
('Green Bowl', 'Surat', 'Healthy');

INSERT INTO menu_items (restaurant_id, item_name, price, category) VALUES
(1, 'Veg Thali', 220.00, 'Main Course'),
(1, 'Paneer Tikka', 280.00, 'Starter'),
(1, 'Gulab Jamun', 80.00, 'Dessert'),
(1, 'Lassi', 60.00, 'Drink'),
(2, 'Margherita Pizza', 350.00, 'Main Course'),
(2, 'Garlic Bread', 160.00, 'Starter'),
(2, 'Brownie', 120.00, 'Dessert'),
(2, 'Lemon Soda', 70.00, 'Drink'),
(3, 'Green Salad', 180.00, 'Starter'),
(3, 'Veg Wrap', 190.00, 'Main Course'),
```

```
(3, 'Fruit Bowl', 200.00, 'Dessert'),
(3, 'Fresh Juice', 100.00, 'Drink');

INSERT INTO orders (customer_name, restaurant_id, item_id, quantity,
total_amount, order_date) VALUES
('Asha',1,1,1,220,'2026-10-01 10:00:00'),
('Bharat',1,2,2,560,'2026-10-01 11:00:00'),
('Chirag',1,3,1,80,'2026-10-02 12:00:00'),
('Diya',1,4,2,120,'2026-10-02 13:00:00'),
('Esha',1,1,2,440,'2026-10-03 14:00:00'),
('Farhan',2,5,1,350,'2026-10-03 15:00:00'),
('Gita',2,6,2,320,'2026-10-04 16:00:00'),
('Harsh',2,7,1,120,'2026-10-04 17:00:00'),
('Isha',2,8,2,140,'2026-10-05 18:00:00'),
('Jay',2,5,2,700,'2026-10-05 19:00:00'),
('Kavya',3,9,1,180,'2026-10-06 10:00:00'),
('Manav',3,10,2,380,'2026-10-06 11:00:00'),
('Nisha',3,11,1,200,'2026-10-07 12:00:00'),
('Om',3,12,2,200,'2026-10-07 13:00:00'),
('Pooja',3,10,1,190,'2026-10-08 14:00:00');

CREATE VIEW restaurant_sales_summary AS
SELECT r.restaurant_id, r.name AS restaurant_name,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.total_amount), 0) AS total_revenue
FROM restaurants AS r
LEFT JOIN orders AS o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name;

DELIMITER //

CREATE PROCEDURE add_order(
    IN p_customer_name VARCHAR(100),
    IN p_restaurant_id INT,
    IN p_item_id INT,
    IN p_quantity INT
)
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_restaurant_count INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;
```

```
    SELECT COUNT(*) INTO v_restaurant_count
    FROM restaurants
    WHERE restaurant_id = p_restaurant_id;

    IF v_restaurant_count = 0 OR p_quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Restaurant not found or quantity is invalid';
    END IF;

    SELECT price INTO v_price
    FROM menu_items
    WHERE item_id = p_item_id
      AND restaurant_id = p_restaurant_id;

    IF v_price IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Menu item not found for this restaurant';
    END IF;

    INSERT INTO orders
        (customer_name, restaurant_id, item_id, quantity, total_amount)
    VALUES
        (p_customer_name, p_restaurant_id, p_item_id, p_quantity,
         v_price * p_quantity);

    COMMIT;
END//

CREATE TRIGGER after_order_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    INSERT INTO order_audit (order_id, restaurant_id, action, log_time)
    VALUES (NEW.order_id, NEW.restaurant_id, 'INSERT', NOW());
END//

DELIMITER ;

SELECT * FROM restaurant_sales_summary;

CALL add_order('Ravi', 1, 1, 2);
SELECT * FROM orders ORDER BY order_id DESC LIMIT 3;
SELECT * FROM order_audit ORDER BY audit_id DESC LIMIT 5;
```

Section D – AI-Augmented Learning

Step 1: BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a stored procedure that:

Accepts a restaurant_id and a month number (1–12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month.

Handles the case where the restaurant has no orders in that month by returning a meaningful message rather than an empty result.

Uses a SELECT with GROUP BY and aggregate functions to compute the summary values correctly.

Runs successfully in MySQL Workbench — screenshot output for at least two different restaurant_id and month combinations.

Exact prompt (example):

```
Write a simple MySQL 8.x stored procedure that accepts a restaurant_id and a
month number (1–12), and returns total orders, total revenue, and average
order value for that restaurant in that month. If there are no orders, return
a meaningful message. Use GROUP BY and aggregate functions. Explain the code
in beginner-friendly language.
```

Step 2: TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the stored procedure and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself.

```
DELIMITER //

CREATE PROCEDURE monthly_restaurant_summary(
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    SELECT COUNT(*) AS total_orders,
           SUM(total_amount) AS total_revenue,
           AVG(total_amount) AS average_order_value
    FROM orders
```

```
    WHERE restaurant_id = p_restaurant_id
      AND MONTH(order_date) = p_month
      AND YEAR(order_date) = YEAR(CURDATE())
    GROUP BY restaurant_id;
END//

DELIMITER ;
```